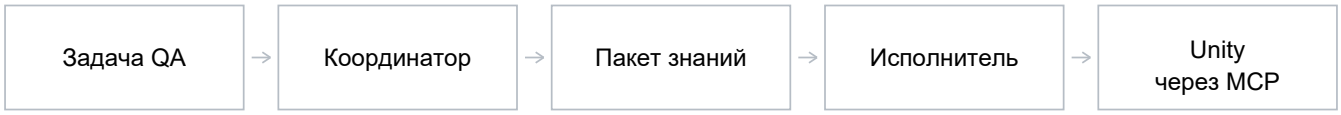
Автоматизация QA через Unity MCP

Архитектура управления и развитие системы
27 октября 2026 года

Мы создали рабочую основу автоматизации тестирования с ИИ в Unity Editor: подключение к игре через MCP, запись демонстраций QA, библиотеку исполняемых действий и внешнюю память проекта. Повторяемые сценарии можно готовить заранее и запускать последовательно одним исполнителем.

Ожидаемый эффект - сократить активное участие тестировщика в повторных проверках. Отдельные компоненты уже работают в игре; полностью автономный регрессионный контур и кратное сокращение общего ручного времени ещё предстоит подтвердить измерениями.

Как проходит запрос



Координатор получает цель, выбирает сценарий и его критерий результата. Из памяти извлекается короткий пакет необходимых знаний. Назначенный исполнитель запускает подготовленные команды, получает свежее состояние Unity и возвращает итог с доказательствами.

Роли и ответственность

- Координатор управляет очередью задач, применимостью знаний и передачей управления. Известная задача использует готовую точку запуска без нового поиска решения.
- Специалисты помогают разобрать новую механику, демонстрацию или конкретный пробел. Их роли описаны отдельно; знакомый повтор обычно не требует дополнительного агента.
- Один исполнитель владеет игровым вводом. Это исключает конкурирующие клики и удержания от нескольких агентов. QA задаёт ожидания, корректирует поведение и подтверждает область применимости результата.

Как команды доходят до редактора

В текущем прототипе JavaScript организует вызовы и очередь, MCP передаёт структурированные запросы JSON, а C# команды и Unity адаптер выполняют действия и наблюдение в Editor. Ответ содержит данные о состоянии, исходе действия и ошибках. MCP предоставляет доступ к инструментам; логику выбора задач, память и критерии проверки мы строим поверх него.

Как устроена внешняя память

Наш «мозг» состоит из модели, структурированных знаний и исполняемых навыков. Обучение в этом проекте означает накопление проверяемого опыта во внешних файлах и коде. Веса модели при этом не изменяются; новый чат использует сохранённые материалы проекта.

- Индекс хранит ID, названия, синонимы, статусы и ссылки. Он помогает выбрать тему без чтения всей переписки и всех карточек.
- Карточки содержат правила, зависимости, ограничения, источники и область проверки. Общий факт имеет одного владельца, а другие карточки ссылаются на него.
- Пакет задачи включает только нужные сведения, предусловия, точку запуска и критерии успеха и остановки. Текущий поиск работает по явным ID, алиасам и метаданным; семантический поиск ещё не внедрён.
- Рабочее состояние получается из игры заново: актуальные объекты, положение, доступность действий и прогресс. Видео и подробные журналы сохраняются отдельно как доказательства конкретной попытки.

Код, правила и отобранные карточки хранятся в репозитории автоматизации. Сырые записи и рабочие кеши остаются локально. Codex и Cursor обмениваются этими файлами; история чатов между ними автоматически не синхронизируется.

Чем навык отличается от миссии

Навык описывает, как выполнить переиспользуемое действие: открыть окно, переместиться, навести камеру или взаимодействовать с объектом. Миссия задаёт цель, порядок применения навыков и ожидаемый результат. Один навык используется в разных миссиях; новый сценарий не должен порождать копию уже изученного способа действия.

Как мы приобретаем новый навык

QA показывает действие и объясняет намерение. Мы сопоставляем объяснение с записью, данными редактора и при необходимости исходниками. Из этого формируем семантическую цель, условия запуска и проверяемый итог. Затем выполняем пробу, разбираем отклонения и после подтверждения сохраняем знание у его владельца. Фиксируются также неудачи и вмешательства человека.

Для чего создан QA Demonstration

QA Demonstration - наш отдельный расширяемый Unity пакет рекордера. Одна кнопка связывает видео Game View и журнал действий; можно записывать только журнал. Доступны время, кадры, сцены и телеметрия поддержанных адаптером объектов, игрока, камеры и квестов. Предусмотрен облегчённый профиль записи.

Демонстрация служит материалом для обучения и проверки, а готовность навыка подтверждается отдельным исполнением. Автоматическое превращение любой записи в надёжный тест пока не реализовано. Новые виды телеметрии можно добавлять модулями пакета; полнота записи зависит от адаптера игры.

Как сокращаем задержки и обращения

Подготовка отделена от выполнения

До прогона подбираются пакеты, загружаются зависимости, компилируются команды и готовятся наблюдатели. Пул сохраняет готовые исполнители по задаче, эпохе Unity и ревизии знаний. При повторе используем тот же подготовленный набор, если среда и исходники совместимы.

Файловый кеш знаний и прогретый исполнитель - разные уровни готовности. Первый хранится на диске, второй живёт в процессе и Unity. Готовые обработчики ждут задания без фоновых обращений к модели; после перезапуска процесса или перезагрузки домена их нужно подготовить снова.

Частое управление выполняет готовый код

Внутри знакомого действия короткие переходы, ожидания и ограниченные повторы выполняет подготовленный обработчик. Модель выбирает цель, подключается при неизвестной ситуации и помогает разобрать результат. Между каждым движением и нажатием не требуется заново рассуждать или загружать память.

Один запуск навыка может содержать несколько технических MCP запросов. Ускорение даёт сокращение лишних решений модели, наблюдений и повторной компиляции. Вместо полного скриншота или иерархии на каждом шаге читается компактное состояние нужных объектов; подробная диагностика включается при отклонении.

Качество сохраняем проверкой результата

- Успех подтверждается свежим состоянием или событием текущей попытки. Отправленная команда и доступная кнопка сами по себе не означают успешный тест.
- Изменение исходников делает старый пакет недействительным до проверки. Координаты, разрешения на действия, исключения ошибок и прежние результаты не переносятся в новый запуск как постоянные знания.
- Неожиданная ошибка, неизвестный исход или команда «стоп» прекращают оставшуюся цепочку. Подробный журнал остаётся локально; координатор получает короткий итог и ссылку на доказательства.

Что подтверждено сейчас

Работают выборочная файловая память, проверка ревизий, каталог исполнителей и последовательный запуск подготовленных задач. В последнем прогоне два новых обработчика завершились примерно за 7,4 и 7,6 секунды каждый вместе с подготовкой игрового состояния; QA одобрил скорость. Другой обработчик потребовал подсказки, после которой исправление закреплено и проверено офлайн. Полная расширенная цепочка одним автономным прогоном ещё не подтверждена.

Что улучшить и какой эффект получить

Ближайшие улучшения

• Устойчивость исполнения. Завершить общий запуск без вмешательств, разделить ошибки игры и инфраструктуры, проверять восстановление после разрыва связи и отмену ввода. Прогон должен завершаться понятным результатом и полным отчётом.

• Наблюдаемость. Унифицировать снимок состояния и события, расширить телеметрию важных проверок. Где возможно, ждать конкретное событие вместо частого опроса и фиксированной длинной паузы. Измерять выборку памяти, прогрев, MCP, ожидания и разрывы между вызовами отдельно.

• Повторное использование. Стабилизировать API навыков, подготовку исходного состояния и версионирование знаний. Продолжить разделение миссий и общих действий. Добавить переносимую установку и экспорт нужных доказательств между компьютерами.

• Организация агентов. Использовать специалиста для конкретного пробела и передавать только нужный пакет. Проверить работу описанных ролей в новых чатах и измерить эффект: увеличение числа агентов само по себе не гарантирует ускорения.

Как применять подход в QA

После стабилизации на этой основе можно собирать smoke проверки сборки, повторные UI и функциональные проверки, сценарии сохранения и восстановления состояния. Один навык будет обслуживать несколько тестов. При сбое можно автоматически собирать контекст и готовить краткий черновик баг репорта для проверки QA.

Следующий уровень - запуск выбранного набора на выделенной машине по готовности сборки, сравнение результатов между версиями и отчёт об изменениях. Это план развития: запуск без оператора и включение проверок в процесс сборки пока не настроены. Перенос на другой проект потребует адаптера и проверки его механик.

Как подтвердить сокращение ручного времени

Для следующего пилота предлагается выбрать 10-15 частых проверок и сравнить одинаковый набор вручную и через автоматизацию. Измерять активное время QA, полную длительность, долю запусков без помощи, повторяемость результата, ложные срабатывания и обнаружение заранее известных дефектов. В расчёт включить первоначальную подготовку, поддержку и вычисления.

Кратное сокращение возможно за счёт уменьшения участия человека в каждом повторе и повторного использования навыков. Достигнутый коэффициент нужно считать по измерениям; сопоставимого ручного базового замера пока нет. Editor контур охватывает доступную в нём логику. Поведение мобильной платформы и производительность релизной сборки требуют отдельной проверки на целевой среде.

Основание отчёта: текущий код исполнителей и памяти, протокол ролей, пакет QA Demonstration, локальные результаты прогонов и подтверждения QA на 27.10.2026. Подробные внутренние ссылки находятся в редактируемом исходнике.